Javascript

AULA 05

PROFESSOR MAROMO

Javascript

Funções - 2ª Parte

Nesta aula veremos outros conceitos que envolvem funções

Funções

- Relembrando
  - Declaração, chamada e argumentos
- Retorno de valores
- Escopo de funções
- Funções anônimas e arrow functions

Funções

Como vimos na aula passada as funções são blocos de código que podem ser definidos e reutilizados para executar tarefas específicas.

Relembrando: Estrutura das funções

```
function nomeDaFuncao(parametro1, parametro2) {
  // Corpo da função - Bloco de código executado quando a função é chamada
  // Pode conter instruções, cálculos, lógica, estruturas de controle e muit
  // Pode também retornar um valor usando a instrução 'return'
}
```

Exemplo fatorial

javascript

```javascript
function calcularFatorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  } else {
    return n * calcularFatorial(n - 1);
  }
}

// Chamando a função para calcular o fatorial de 5
let resultado = calcularFatorial(5);
console.log(resultado);  // Resultado: 120
```

Escopo de Funções

Antes de prosseguir com os detalhes sobre o escopo de funções em JavaScript, é importante entender a importância desse conceito no desenvolvimento de programas. O escopo desempenha um papel fundamental na organização e no controle de acesso às variáveis em um programa.

Escopo de Funções

Ao entender o escopo, você adquire a capacidade de escrever código mais eficiente, legível e livre de erros.

Razões para compreender

1. Evitar conflitos de nomes: Ao dividir um programa em diferentes funções, é provável que você precise usar variáveis com o mesmo nome em contextos diferentes. Compreender o escopo permite evitar conflitos de nomes, garantindo que as variáveis não se sobreponham indevidamente.
2. Encapsulamento de variáveis: O escopo permite encapsular variáveis dentro de funções. Isso significa que essas variáveis só são acessíveis dentro das funções em que foram declaradas, mantendo-as protegidas e evitando interferências indesejadas de outras partes do código.

Razões para compreender

3. Reutilização de nomes: Entender o escopo permite reutilizar nomes de variáveis em diferentes partes do código sem gerar conflitos. Como as variáveis declaradas em um escopo local não são acessíveis em escopos externos, você pode usar os mesmos nomes sem problemas.
4. Segurança e privacidade de dados: Ao limitar o acesso às variáveis em escopos específicos, você aumenta a segurança e a privacidade de dados. Variáveis sensíveis podem ser declaradas em escopos locais e acessíveis apenas por funções autorizadas.

Razões para compreender

5. Otimização de memória: O escopo também está relacionado à eficiência do programa. Quando uma função é concluída, as variáveis locais dentro dela são liberadas da memória. Compreender o escopo permite otimizar o uso da memória, garantindo que variáveis desnecessárias sejam liberadas corretamente.

Locais e Globais

Variáveis globais e locais: No escopo **global**, as variáveis são declaradas fora de qualquer função e podem ser acessadas em qualquer lugar do código. No escopo **local**, as variáveis são declaradas dentro de uma função e só podem ser acessadas dentro dessa função.

Locais e Globais

javascript

```javascript
let nome = "João"; // Variável global

function saudacao() {
  let mensagem = "Olá"; // Variável local
  console.log(mensagem + ", " + nome); // Acessa variáveis t
}

saudacao(); // Resultado: "Olá, João"
```

Locais e Globais

javascript

```javascript
let nome = "João"; // Variável global

function saudacao() {
  let mensagem = "Olá"; // Variável local
  console.log(mensagem + ", " + nome); // Acessa variáveis t
}

saudacao(); // Resultado: "Olá, João"
```

Locais e Globais

Nesse exemplo, a variável **nome** é uma variável global, pois foi declarada fora da função. Ela pode ser acessada tanto dentro da função saudacao quanto fora dela. Já a variável **mensagem** é uma variável local, pois foi declarada dentro da função saudacao e só pode ser acessada dentro dessa função.

Escopo aninhado

Quando uma função é declarada dentro de outra função, ela cria um escopo aninhado.
As variáveis declaradas na função externa podem ser acessadas pela função interna, mas não o contrário.

javascript

```javascript
function calcularIdade() {
  let idade = 25; // Variável na função externa

  function exibirIdade() {
    console.log(idade); // Acessa a variável da função externa
  }

  exibirIdade(); // Resultado: 25
}

calcularIdade();
```

Sombreamento

Sombreamento de variáveis: Quando uma variável é declarada em um escopo local com o mesmo nome de uma variável em um escopo externo, ocorre o sombreamento da variável externa. Isso significa que a variável local tem prioridade sobre a variável externa dentro do escopo local.

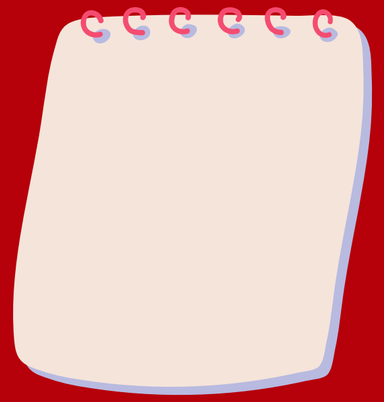
javascript Copy code

```javascript
let nome = "Maria"; // Variável global

function saudacao() {
  let nome = "João"; // Variável local com o mesmo nome

  console.log("Olá, " + nome); // Acessa a variável local
}

saudacao(); // Resultado: "Olá, João"
console.log("Nome global: " + nome); // Resultado: "Nome global: Maria"
```

Sombreamento

Nesse exemplo, a função saudacao possui uma variável local chamada nome, que sombreia a variável global de mesmo nome. Dentro da função, a variável local é usada para exibir a saudação, enquanto fora da função, a variável global é acessada.

Nota:



Compreender o escopo de funções em JavaScript é fundamental para escrever código organizado, evitar conflitos de nomes, proteger variáveis, reutilizar nomes de forma segura e otimizar a memória do programa.

Diferença entre let e var

Escopo de bloco

- **let:** A declaração let introduz uma variável com escopo de bloco, o que significa que ela é visível apenas dentro do bloco onde foi declarada. Um bloco é delimitado por chaves {}. Variáveis declaradas com **let** têm escopo limitado ao bloco em que foram definidas, incluindo loops, condicionais e funções.
- **var:** A declaração var, por outro lado, tem escopo de função ou escopo global. Isso significa que uma variável declarada com **var** é visível dentro da função onde foi declarada ou, se não estiver dentro de uma função, é visível globalmente.

let

```
function exemplo() {
  if (true) {
    let x = 10; // Escopo de bloco
    console.log(x); // 10
  }
  console.log(x); // ReferenceError: x is not defined (não está acessível
}
```

var

```
function exemplo() {
  if (true) {
    var y = 20; // Escopo de função
    console.log(y); // 20
  }
  console.log(y); // 20 (ainda está acessível fora do bloco)
}
```

Escopo de bloco

- Nesses exemplos, a variável x é declarada com let e é acessível apenas dentro do bloco if. Já a variável y é declarada com var e é acessível tanto dentro quanto fora do bloco if.

Redeclaração

let: A redeclaração de uma variável com let no mesmo escopo não é permitida. Tentar redeclarar uma variável com let resultará em um erro.
var: A redeclaração de uma variável com var no mesmo escopo é permitida, mas pode levar a comportamentos inesperados, pois a variável será substituída pela nova declaração.

Redeclaração

```
let x = 10;
let x = 20; // SyntaxError: Identifier 'x' has already been declared
```

```
var y = 10;
var y = 20; // Sem erro, mas a segunda declaração substitui a primeira
console.log(y); // 20
```

No exemplo com let, ocorre um erro de sintaxe ao tentar redeclarar a variável x. Já com var, a redeclaração não gera um erro, mas substitui o valor da variável pela nova atribuição.

Escopo em loop

let: Quando uma variável é declarada com let em um loop (como for ou while), ela terá escopo de bloco, o que significa que cada iteração do loop terá uma nova instância da variável. Isso pode ser útil em situações onde você precisa armazenar valores diferentes em cada iteração do loop.

var: A variável declarada com var em um loop tem escopo de função ou escopo global. Isso significa que a mesma instância da variável é compartilhada entre todas as iterações do loop.

Exemplos

javascript Copy code

```javascript
for (let i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i); // 0, 1, 2 (cada iteração cria um novo escopo)
  }, 1000);
}
```

```javascript
for (var j = 0; j < 3; j++) {
  setTimeout(function() {
    console.log(j); // 3, 3, 3 (mesma instância compartilhada)
  }, 1000);
}
```

Explicação

No exemplo com **let,** cada iteração do loop cria um novo escopo para a variável i, permitindo que cada função de retorno de setTimeout acesse o valor correto de i em sua respectiva iteração.

Já no exemplo com **var**, a mesma instância de j é compartilhada entre todas as funções de retorno de setTimeout, resultando no mesmo valor para todas as iterações.

Exercícios para prática

Temos os enunciados corretos para as 5 questões que envolvem o uso de **let** e **var**.

Resolva:

EXERCÍCIO 1

javascript Copy code

```javascript
function exercise1() {
  let x = 10;
  if (x > 5) {
    let x = 20;
    console.log(x); // Qual será o valor exibido no console dentro do bloco
  }
  console.log(x); // Qual será o valor exibido no console fora do bloco if?
}
```

EXERCÍCIO 2

javascript Copy code

```javascript
function exercise2() {
  var x = 10;
  if (x > 5) {
    var x = 20;
    console.log(x); // Qual será o valor exibido no console dentro do bloco
  }
  console.log(x); // Qual será o valor exibido no console fora do bloco if?
}
```

EXERCÍCIO 3

javascript Copy code

```javascript
function exercise3() {
  for (let i = 0; i < 3; i++) {
    setTimeout(function() {
      console.log(i); // Quais valores serão exibidos no console?
    }, 1000);
  }
}
```

EXERCÍCIO 4

javascript Copy code

```javascript
function exercise4() {
  for (var i = 0; i < 3; i++) {
    setTimeout(function() {
      console.log(i); // Quais valores serão exibidos no console?
    }, 1000);
  }
}
```

EXERCÍCIO 5

javascript Copy code

```javascript
function exercise5() {
  let y = 5;
  var y = 10; // Haverá um erro de declaração ou algum comportamento inesper
  console.log(y); // Qual será o valor exibido no console?
}
```

Agora você pode tentar resolver esses exercícios com base nos enunciados. Depois, fornecerei as soluções para você conferir suas respostas.

FIM
Seja a mudança que você deseja ver no mundo, impactando positivamente a vida de outras pessoas por meio das suas ações!